

Telco Customer Churn Prediction using Deep Learning (DNN)

The Telco customer churn data contains information about a fictional telco company that provided home phone and Internet services to 7043 customers in California in Q3. It indicates which customers have left, stayed, or signed up for their service. Multiple important demographics are included for each customer, as well as a Satisfaction Score, Churn Score, and Customer Lifetime Value (CLTV) index.

This dataset is a gold standard for business analytics because it allows for high-impact Deep Learning applications (Multi-Layer Perceptrons) that directly correlate to revenue retention.

Business Objective & Modeling Focus

Main Objective

The primary objective of this analysis is to predict customer churn using Deep Learning (Fully Connected Neural Networks) and to compare multiple DNN architectures to identify the best trade-off between predictive accuracy, stability, and business interpretability.

Business Benefits

- Accurate churn prediction enables the business to:
- Proactively identify high-risk customers
- Design targeted retention campaigns
- Reduce customer acquisition costs
- Improve Customer Lifetime Value (CLV)

Project Overview: Deep Learning for Retention

Algorithm: Deep Neural Networks (Multi-Layer Perceptron / MLP).

Objective: To build a binary classifier that predicts if a customer will leave (Churn=Yes) or stay (Churn=No).

Business Benefit: Reducing churn is up to 5x cheaper than acquiring new customers. This model identifies at-risk subscribers, allowing the marketing department to offer proactive retention incentives.

Dataset Description:

The dataset Contains 7,043 rows of customer data with 21 attributes including demographics (age, gender), account information (tenure, contract type), and services (internet, streaming).

Category	Examples
Customer Info	gender, SeniorCitizen
Services	InternetService, OnlineSecurity
Contract & Billing	Contract, MonthlyCharges
Tenure	tenure
Target	Churn

Libraries and Configuration

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

Load and inspect Data

```
In [5]: df = pd.read_csv("Telco-Customer-Churn.csv")
df.head()
```

Out[5]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	No

5 rows × 21 columns



In [6]: `df["Churn"].value_counts(normalize=True)`

Out[6]:

```
Churn
No    0.73463
Yes   0.26537
Name: proportion, dtype: float64
```

Data Cleaning & Feature Engineering

Cleaning Steps:

- Convert TotalCharges to numeric
- Drop customerID
- Encode categorical variables
- Scale numerical features

In [7]:

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df.dropna(inplace=True)
df.drop(columns=["customerID"], inplace=True)
```

Encode Target

In [8]:

```
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})

df = pd.get_dummies(df, drop_first=True)
```

Split - Train - Scale Dataset

```
In [9]: X = df.drop("Churn", axis=1)
y = df["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Define Deep Learning Architecture

Baseline Neural Network

```
In [10]: model_1 = Sequential([
    Dense(32, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(1, activation="sigmoid")
])

model_1.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history_1 = model_1.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)
```

c:\Users\jpdob\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:95: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Deep Neural Network with Dropout

```
In [11]: model_2 = Sequential([
    Dense(64, activation="relu", input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid")
])
```

```

model_2.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history_2 = model_2.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)

```

Regularized Neural Network

```

In [12]: early_stop = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

model_3 = Sequential([
    Dense(128, activation="relu", input_shape=(X_train.shape[1],)),
    Dropout(0.4),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])

model_3.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history_3 = model_3.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)

```

Evaluate all three models

```

In [13]: def evaluate_model(model, X_test, y_test):
    y_pred_prob = model.predict(X_test).ravel()
    y_pred = (y_pred_prob > 0.5).astype(int)
    return {
        "Accuracy": accuracy_score(y_test, y_pred),
        "ROC_AUC": roc_auc_score(y_test, y_pred_prob)
    }

```

```

    }

results = pd.DataFrame([
    evaluate_model(model_1, X_test, y_test),
    evaluate_model(model_2, X_test, y_test),
    evaluate_model(model_3, X_test, y_test)
], index=["Baseline NN", "Deep NN + Dropout", "Regularized NN"])

results

```

```

44/44 ————— 0s 2ms/step
44/44 ————— 0s 2ms/step
44/44 ————— 0s 2ms/step

```

Out[13]:

	Accuracy	ROC_AUC
Baseline NN	0.782516	0.831221
Deep NN + Dropout	0.779673	0.821803
Regularized NN	0.797441	0.832058

Recommended Final Model

Selected Model: Model 3 – Regularized Neural Network

Why?

- Best ROC-AUC performance
- Controlled overfitting via dropout + early stopping
- More stable during training
- Suitable for production deployment
- While slightly more complex, it offers better generalization, which is critical in churn prediction.

Key Findings & Insights

- Customers on month-to-month contracts show significantly higher churn
- Low tenure + high monthly charges strongly correlate with churn
- Deep learning models outperform simple baselines by capturing non-linear interactions
- Regularization is essential due to class imbalance and noisy categorical features

Next Steps & Recommendations

Suggested Enhancements

- Cost-sensitive learning (penalize false negatives more)
- Explainability using SHAP or Integrated Gradients

- Compare with Gradient Boosting + DNN hybrid

Conclusion

This Deep Learning–based churn framework demonstrates how neural networks can be effectively applied to structured business data, providing actionable insights and measurable business value.